Distributed Systems Handin 3

BSDISYS1KU Carl Philip Steuch caps@itu.dk & Johannes Jensen johje@itu.dk 29-10-2025

https://github.com/Joha6210/DistributedSystems-Handin3

1. Streaming Model Selection

1.1 Discussion

- *Discuss, whether you are going to use server-side streaming, client-side streaming, or bidirectional streaming?*
- In our chit chat implementation we use server-side streaming (1), to send multiple messages from the server to the client. This is done to support long-lived logical flow of data from the server to the clients (2)

---

2. System Architecture

2.1 Overview

- **Architecture Type:**
  Chit chat is implemented as a server-client architecture, this allows for the server to serve multiple clients that can connect concurrently. Clients can leave and join whenever they like, and they will get the chat history send to them upon subscribing(connecting) to the server.

2.2 Components

- **Server:** The server is responsible for letting clients 'subscribe', publish messages to chit chat and unsubscribe when the client wants to leave. The server also keeps track of the message history. The server will announce when a new client either joins or leaves the server.
- **Client:** Clients can connect to the chit chat server using the 'subscribe' method, this is for the client to "register" at the server, and allows for the client to receive the message history and new messages that are being published by other clients. Registered clients can also publish new messages to the chit chat server, for other clients to receive.

3. RPC Methods and Message Types

3.1 Implemented RPC Methods

RPC Method	Type (Unary/Server Streaming/Client Streaming/Bidirectional)	Description
PublishMessage (Message)	Unary	Publishes Message object to server
Subscribe (Client)	Server Streaming	Adds client to server
Unsubscribe (Client)	Unary	Removes client from server

3.2 Message Types

Message Name	Purpose	Fields
message Message	Contains the message and relevant information	`string uuid = 1; string message = 2; int32 clock = 3; string username = 4; string timestamp = 5;`
message Response	Contains a response from the server to a client	`bool result = 1; int32 clock = 3;`
message Client	Contains information about a client	`string uuid = 1; string username = 2; int32 clock = 3;`

4. Lamport Timestamp Implementation

Anytime a client or a server receives a message, or is about to initialize an internal process, it's internal Lamport Timestamp(LT) is incremented by 1.

To keep parity across participants, any communication between parties include the originators current LT, and the receiver adopts the larger between the received value and it's own

on initialization do

```
t := 0  // each node has its own local variable t
end on
```

on request to send message m do

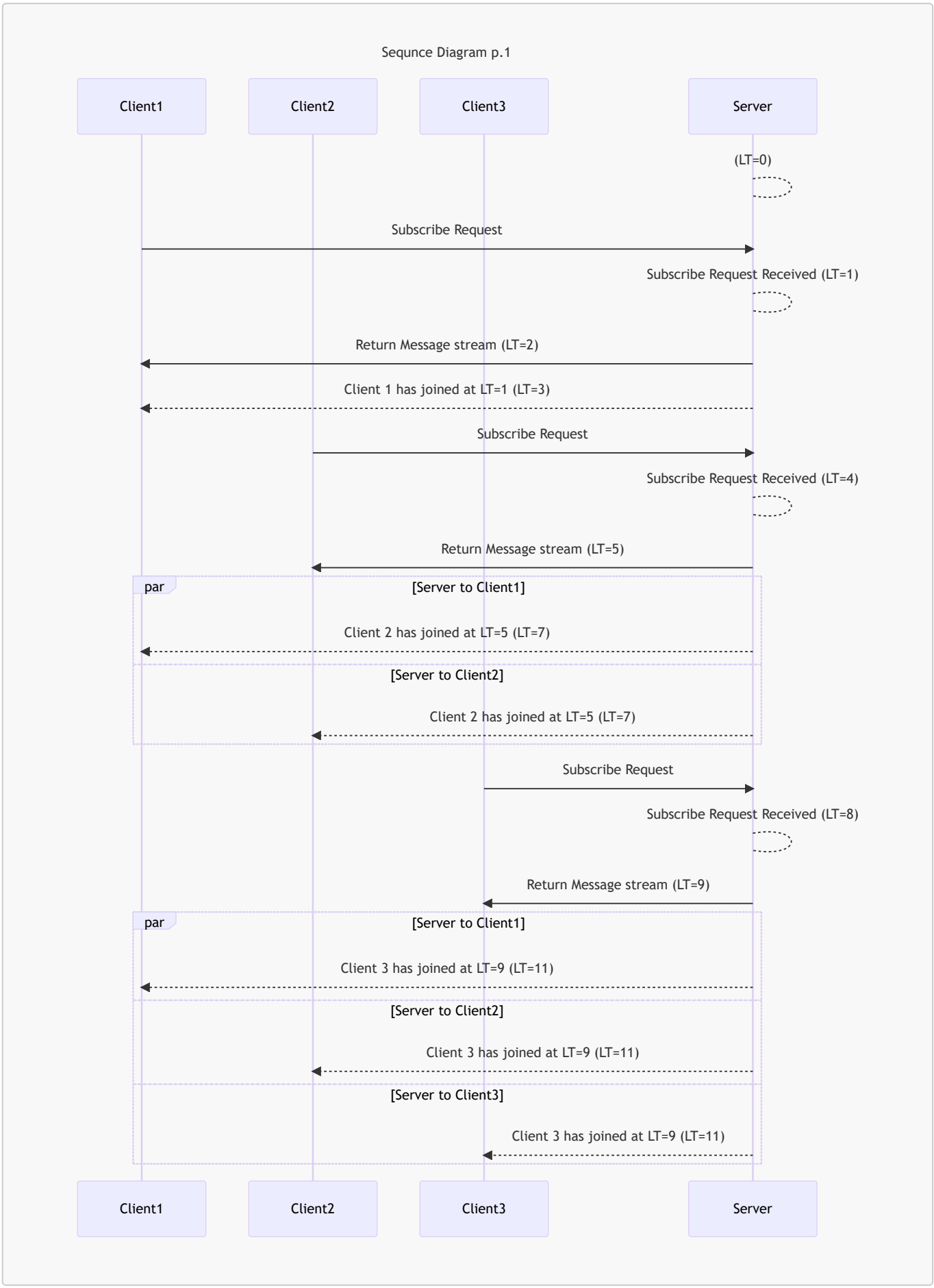
```
t := t + 1; send (t, m) via the underlying network link
end on
```

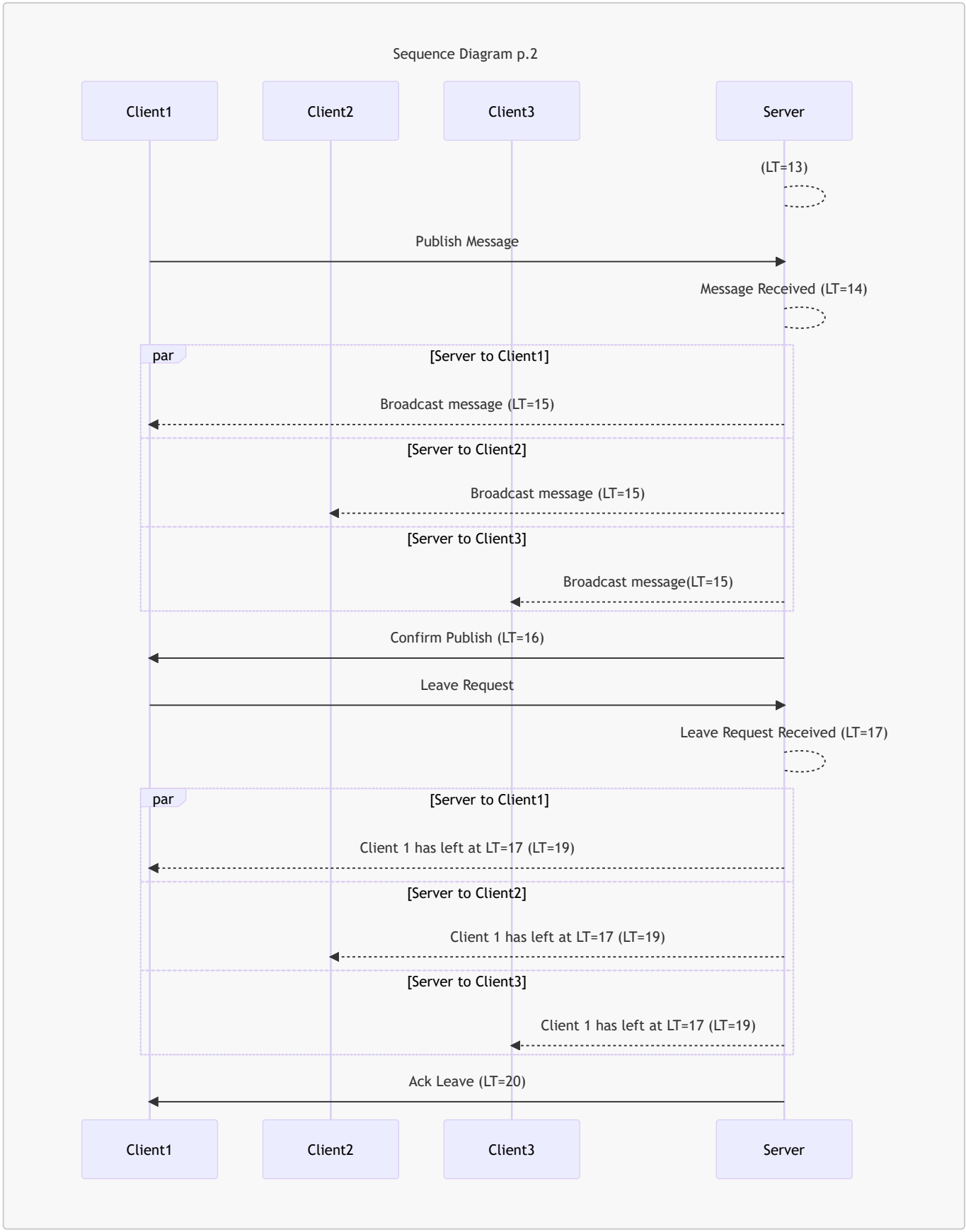
on receiving (t',m) via the underlying network link do

```
t := max(t, t') + 1
 deliver m to the application
end on
```

5. Sequence Diagram

LT refers to local server time, and is assumed to initialize at LT=0 for all parties.




Sequence Diagram p.2
Client1
Client2
Client3
Server
(LT=13)
Publish Message
Message Received (LT=14)
par
[Server to Client1]
Broadcast message (LT=15)
[Server to Client2]
Broadcast message (LT=15)
[Server to Client3]
Broadcast message(LT=15)
Confirm Publish (LT=16)
Leave Request
Leave Request Received (LT=17)
par
[Server to Client1]
Client 1 has left at LT=17 (LT=19)
[Server to Client2]
Client 1 has left at LT=17 (LT=19)
[Server to Client3]
Client 1 has left at LT=17 (LT=19)
Ack Leave (LT=20)
Client1
Client2
Client3
Server